

SMOHIT ARMAN SWAIN

192572092

CSE-AI

## CO3\_AT1\_Application Based Questions

### Q1 Debugging and Rewriting

10 ma

The program below simulates a simple banking system using lists and dictionaries. It has FOUR bugs. Identify each bug with reason and rewrite the correct program.

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] =+ amount
        accounts[acc_no]['txns'].append(("Deposit", amount))
def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] > amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(("Withdraw", amount))
    else:
        print("Insufficient balance")
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5]:
        print(t)
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
```

Your Code

```
1 accounts = {}
2
3 def create_account(acc_no, name, balance):
4     accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
5
6 def deposit(acc_no, amount):
7     if acc_no in accounts:
8         accounts[acc_no]['balance'] += amount
9         accounts[acc_no]['txns'].append(('Deposit', amount))
10
11 def withdraw(acc_no, amount):
12     if accounts[acc_no]['balance'] >= amount:
13         accounts[acc_no]['balance'] -= amount
14         accounts[acc_no]['txns'].append(('Withdraw', amount))
15     else:
16         print('Insufficient balance')
17
18 def mini_statement(acc_no):
19     for t in accounts[acc_no]['txns'][-5:]:
20         print(t)
21
22 create_account(1001, 'Ravi', 10000)
23
24 deposit(1001, 5000)
25
26 withdraw(1001, 20000)
27
28 withdraw(1001, 3000)
```

```
27
28 withdraw(1001, 3000)
29
30 mini_statement(1001)
31
32 print('Balance:', accounts[1001]['balance'])
```

Input

stdin input

Output

```
Insufficient balance
('Deposit', 5000)
('Withdraw', 3000)
Balance: 12000
```

**Q2 Student Grade Book Application**

10 marks

You are building a student grade book application. Write a Python program that:

- (a) Stores each student's name and marks in a dictionary (key = name, value = list of 5 subject marks).
- (b) Computes total, average, and grade (O:  $\geq 90$ , A+:  $\geq 80$ , A:  $\geq 70$ , B+:  $\geq 60$ , B:  $\geq 50$ , F:  $< 50$ ) for each student.
- (c) Finds the class topper (highest average) and the subject with the highest overall mean.
- (d) Displays a formatted result table sorted by average in descending order.

Use dictionary comprehension and list operations. Minimum 5 students.

Write complete, syntactically correct Python programs using lists, tuples, dictionaries, and sets with proper indentation.

**Your Code**

```
1 # Student Grade Book Application
2
3 # Store student names and marks in a dictionary
4 students = {
5     'Ravi': [95, 88, 92, 90, 94],
6     'Anita': [78, 85, 80, 82, 79],
7     'Kiran': [67, 72, 70, 69, 71],
8     'Meena': [55, 60, 58, 62, 57],
9     'Arjun': [89, 91, 87, 90, 88]
10 }
11
12 subjects = ['Sub1', 'Sub2', 'Sub3', 'Sub4', 'Sub5']
13
14 # Function to calculate grade
15 def get_grade(avg):
16     if avg >= 90:
17         return 'O'
18     elif avg >= 80:
19         return 'A+'
20     elif avg >= 70:
21         return 'A'
22     elif avg >= 60:
23         return 'B+'
24     elif avg >= 50:
25         return 'B'
26     else:
27         return 'F'
28
```

```

# Compute total, average, and grade using dictionary comprehension
results = {
    name: {
        'marks': marks,
        'total': sum(marks),
        'average': sum(marks) / len(marks),
        'grade': get_grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}

# Find class topper
topper = max(results.items(), key=lambda x: x[1]['average'])

# Find subject with highest overall mean
subject_means = [
    sum(students[name][i] for name in students) / len(students)
    for i in range(5)
]

highest_subject_index = subject_means.index(max(subject_means))
highest_subject = subjects[highest_subject_index]

# Sort by average descending
sorted_results = sorted(
    results.items(),
    key=lambda x: x[1]['average'],
    reverse=True
)

```

```

56     reverse=True
57 )
58
59 # Display result table
60 print('=' * 85)
61 print(f"{'Name':<10}{'Marks':<30}{'Total':<10}{'Average':<10}{'Grade':<10}")
62 print('=' * 85)
63
64 for name, data in sorted_results:
65     marks_str = ' '.join(str(m) for m in data['marks'])
66     print(f"{name:<10}{marks_str:<30}{data['total']:<10}{data['average']:<10}{data['grade']:<10}")
67
68 print('=' * 85)
69
70 # Display topper
71 print(f"Class Topper : {topper[0]} ({topper[1]['average']:.2f})")
72
73 # Display subject means
74 print('\nSubject Means:')
75 for sub, mean in zip(subjects, subject_means):
76     print(f"{sub}: {mean:.2f}")
77
78 print(f"\nSubject with Highest Overall Mean: {highest_subject} ({max(subject_means.values()):.2f})")

```

## Output

```

Ravi      95 88 92 90 94
459      91.80      O
Arjun     89 91 87 90 88
445      89.00      A+
Anita     78 85 80 82 79
404      80.80      A+

```

```

Kiran     67 72 70 69 71
349      69.80      B+
Meena     55 60 58 62 57
292      58.40      B

```

```
Class Topper : Ravi (91.80)
```

```
Subject Means:
```

```
Sub1: 76.80
```

```
Sub2: 79.20
```

```
Sub3: 77.40
```

```
Sub4: 78.60
```

```
Sub5: 77.80
```

```
Subject with Highest Overall
Mean: Sub2 (79.20)
```